23CSE455 Design Patterns

Case Study Report

Team B15:-
Harikrishna T P (AM.SC.U4CSE23321)
Chandana CM (AM.SC.U4CSE23313)
Pooja T (AM.SC.U4CSE23342)
Rithu Mithra SS (AM.SC.U4CSE23345)

Application: Data-to-Science (D2S)
Source repository: https://github.com/gdslab/data-to-science
Submission repository: https://github.com/aswathymohan-amrita/23CSE455_DP_CaseStudy
Date: 2026-10-11

1 Application and Architecture

1.1 Application description

We studied Data-to-Science (D2S), an open-source geospatial research data management platform. We identified its main domain entities as researchers, teams, projects, campaigns, flights, raw imagery, derived data products, spatial metadata, access permissions, and processing tasks. Through the web interface, researchers can upload large datasets, trigger processing, and explore raster, vector, and point-cloud products. In the supplied source snapshot, we identified React and TypeScript in the frontend and FastAPI/Starlette, SQLAlchemy, PostgreSQL/PostGIS, Celery, and external photogrammetry processing in the backend.

1.2 Environment and source scope

We used the source URL https://github.com/gdslab/data-to-science. Our Part 1 verification note records baseline commit 02230dd745efef089f1f6c30298f0761c17f530c; we preserve that value from the supplied notes. We inspected backend API endpoints, CRUD modules, models, workflow modules, security/static-file handling, frontend API/auth context, documentation, migrations, and vendored assets. We excluded vendored libraries, caches, compiled Python bytecode, environment files, and large unrelated generated assets from our design-pattern evidence. Reported stack: Python 3.12, FastAPI 0.139.0, Starlette 1.3.1, PostgreSQL 16, SQLAlchemy 2.0.51, Alembic 1.18.5, React 18, TypeScript 5, Vite 5, Celery 5.5.3, Redis 5.0+ (versions transcribed from the supplied Part1 PDF.
Version/tag: 02230dd745efef089f1f6c30298f0761c17f530c
License: GitHub repository MIT License.

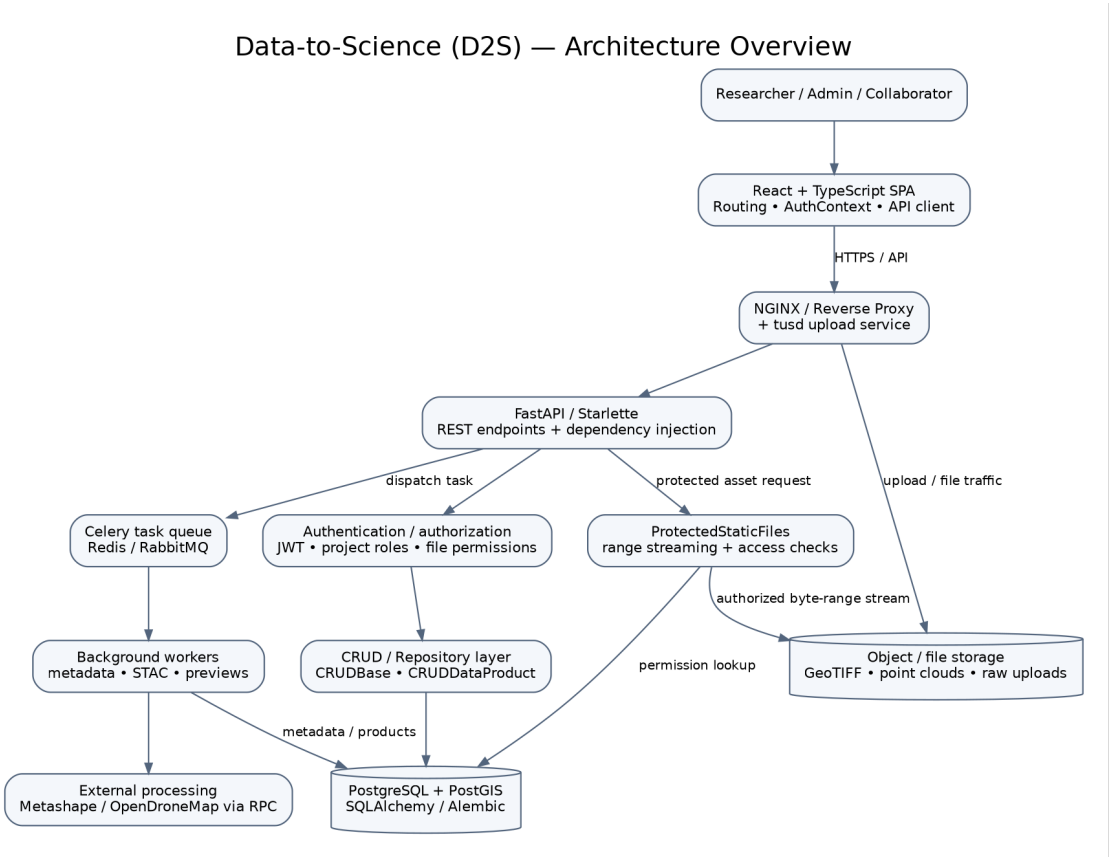


Figure 1: Architecture reconstructed from the supplied source tree and architecture notes.

1.3 Request and event flow

We traced the request flow from the React single-page application through the reverse proxy to FastAPI endpoints. FastAPI dependencies supply authentication context and database sessions. CRUD/repository objects interact with PostgreSQL/PostGIS. Upload completion and processing requests can dispatch Celery tasks; workers extract spatial metadata, generate STAC records and previews, and call external processing engines. The protected static-file wrapper checks access before streaming large files with byte-range responses.

1.4 Run instructions and reproducibility

We used the ZIP's root README to identify sample commands for the Part 3 snippets (Python, Node.js, Java/JDK 17, and C++17). We did not execute those commands in a clean environment. To run the full D2S platform, we would follow its own README and Docker Compose instructions and configure the required services. We should remove secrets and local environment files before committing or redistributing the uploaded ZIP.

2 Part 1 — Pattern Discovery

2.1 Method

We evaluated candidate patterns by inspecting code roles and interactions rather than relying on names alone: abstractions and interfaces, concrete implementations, delegation, subscription and notification, access checks, and repository/database boundaries. We give source locations relative to

`Part1/code/`. We treat a design-pattern label as confirmed only where the supplied snapshot shows a plausible collaboration.

2.2 Pattern inventory

Pattern / idiom	Instance	Category	Evidence in supplied source
Repository / DAO	Generic CRUD persistence	Architectural	`backend/app/crud/base.py`:16–71; `crud/crud_data_product.py`:69+
Dependency Injection	Request-scoped DB/user dependencies	Framework idiom	`backend/app/api/deps.py`; FastAPI Depends use in endpoints
Proxy / Decorator-like wrapper	Protected static file serving	Structural / framework idiom	`backend/app/utils/ProtectedStaticFiles.py`; subclasses RangedStaticFiles and intercepts requests
Strategy	Access policy rules	Behavioral	`backend/app/core/policy_engine.py`:34–111; AccessPolicyRule and concrete rule classes
Chain of Responsibility-like evaluation	Ordered policy evaluation	Behavioral (qualified)	`backend/app/core/policy_engine.py`:114–145; engine iterates rules; verify exact short-circuit semantics before stronger classification
Observer	Workflow status listeners	Behavioral	`backend/app/workflow/base.py`:48–61; `pipeline.py`:23–57
Template Method candidate / workflow orchestration	Ordered workflow execution	Behavioral candidate, not automatically confirmed as GoF Template Method	`backend/app/workflow/pipeline.py`:60–91; execution is concrete orchestration over WorkflowStep
Axios interceptor chain	Refresh/replay after authentication errors	Framework idiom / chain-like	`frontend/src/api.ts`:26–77

2.3 Investigation areas

Area	Observed status	Source-grounded summary
Researcher management	Present	We found user, team, team-member, and project-member models/endpoints in the source tree.
Dataset management	Present	Raw data, data products, uploads, vector layers, and protected file-serving paths are represented.
Experiment management	Present	Projects, campaigns and flights organize research/processing entities.
Metadata	Present	Data-product metadata and STAC/spatial metadata processing appear in models/tasks.
Dataset versioning	Partial / cannot fully confirm	We described a version-history addition in our Part 2 PDF, but we could not fully confirm the CH01 model/CRUD files, version endpoints, or test evidence from our supplied ZIP.
Access control	Present, with extent to verify	Existing roles and file permissions are present; a policy engine is also included in the Part 1 snapshot. JSON-policy integration and end-to-end enforcement need tests/source evidence.

Area	Observed status	Source-grounded summary
Workflow	Present in snapshot	We found ExperimentWorkflowPipeline, WorkflowStep, and listener abstractions in our Part 1 source. We did not include the changed Celery integration and UI files described for CH03.
Notifications	Partial / cannot fully confirm	We found transactional email paths and workflow listener hooks, but we could not establish a complete notification center or deployed WebSocket delivery from our supplied evidence.

2.4 Pattern evidence and UML

In the policy-rule diagram, we map AccessPolicyRule to concrete rules and show the engine evaluating them. In the workflow diagram, we map the pipeline, listener abstraction, context, and workflow steps. We based both diagrams on files present in our ZIP.

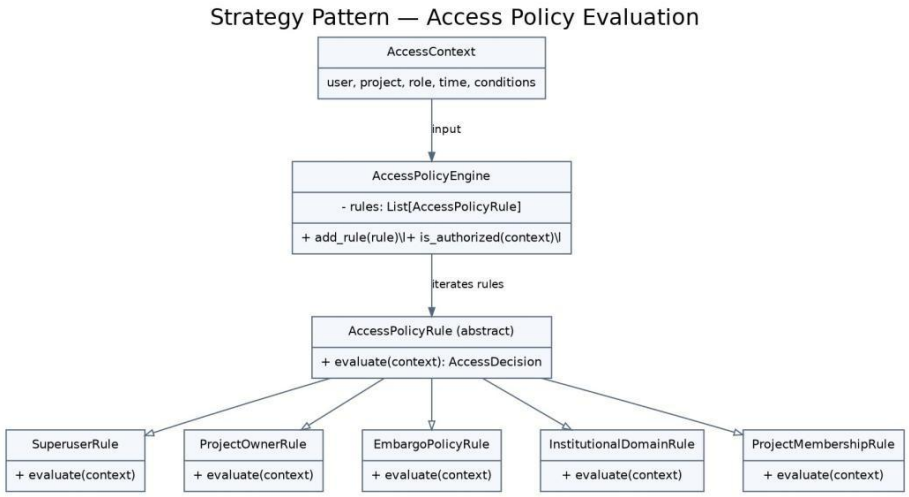


Figure 2: Strategy and policy-evaluation collaboration.

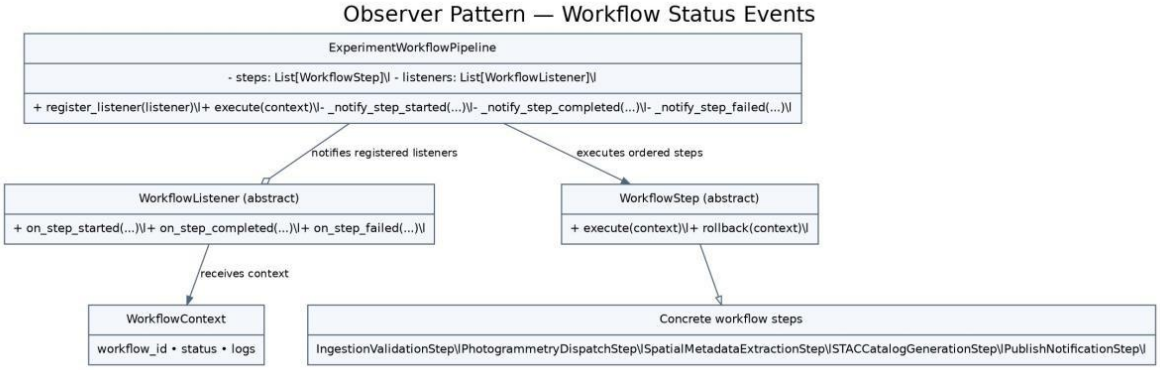


Figure 3: Workflow pipeline and observer/listener collaboration.

3 Part 2 — Software Evolution

We organize the evolution analysis around the three changes specified in the assignment: CH01 dataset versioning, CH02 advanced access control, and CH03 workflow automation. In our supplied `Part2_Evolution.pdf`, we describe requirements, acceptance criteria, pattern effects, refactorings, and metrics for all three.

CH01 — Dataset versioning

In the supplied PDF, we describe a DataProductVersion model, automatic snapshots on update, and version-list/retrieval endpoints. We also describe extending the repository collaboration and extracting update validation. However, we could not find the DataProductVersion model, version endpoints, CH01 diff, or test log in the supplied ZIP. We therefore label the reported LOC, latency, and coverage values as *reported claims*, not independently reproduced measurements.

CH02 — Advanced access control

We inspected the supplied policy_engine.py, which defines AccessContext, AccessDecision, an abstract AccessPolicyRule, concrete rules, and AccessPolicyEngine. This supports Strategy-like policy implementations and ordered evaluation. However, we found identical byte content in the Part 1 and Part 2 copies (SHA-256 cd31565155088b70e877846237bffa1c944a5612d244404cb3264e889e10cef6). Thus, we cannot establish a before/after source change for CH02 from this ZIP. Our Part 2 note says the code was integrated and tests passed.

CH03 — Experiment workflow automation

In the PDF, we describe a workflow pipeline, Celery tasks, UI status display, observer notifications, and a Template Method refactoring. Our supplied Part 1 snapshot already contains ExperimentWorkflowPipeline, listener callbacks, and workflow-step abstractions. We could not find the CH03-specific frontend component, changed Celery task files, diff, or test log in the archive. We can document the change from the PDF.

3.1 Before/after and refactoring summary

Change	Intended effect	Pattern effect claimed in PDF	Evidence status in ZIP
CH01	Historical dataset snapshots and retrieval	Repository extended	Narrative only; implementation/test evidence absent
CH02	Centralized role/attribute policy evaluation	Strategy + Chain of Responsibility	Policy code exists, but Part 1/Part 2 copies identical; integration/diff/tests absent
CH03	Automated workflow and status notifications	Observer + Template Method	Pipeline/listeners exist in Part 1; CH03-specific changed files/logs absent

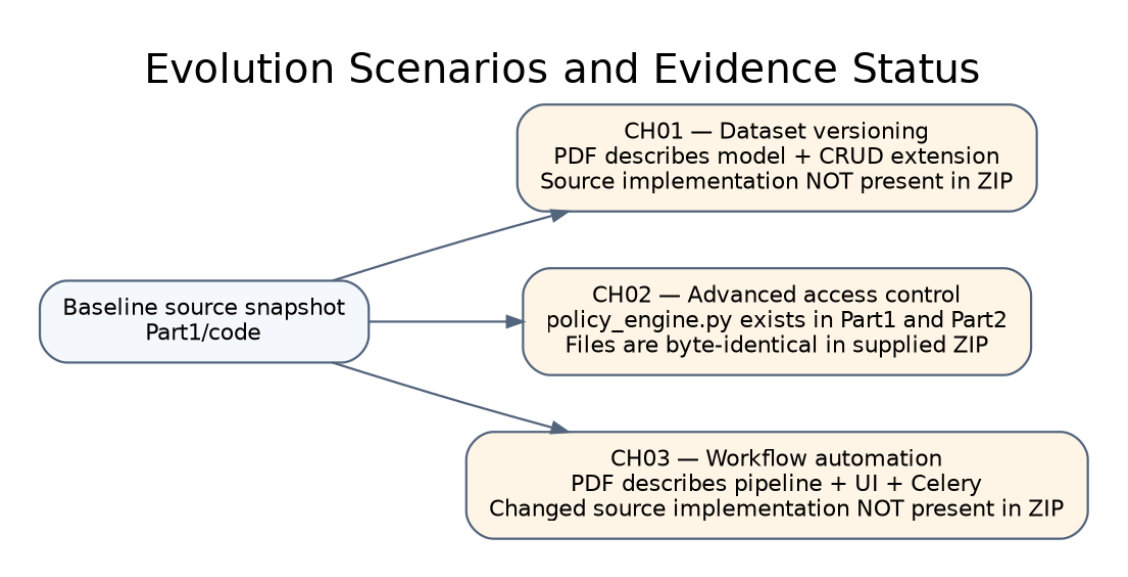


Figure 4: Evolution scenarios and the evidence available in the supplied archive.

4 Part 3 — Cross-Language Analysis

4.1 Selected patterns and shared problem

We selected Strategy (choosing a policy implementation for an allow/deny decision) and Observer (notifying multiple listeners as a workflow moves through STARTED, PROCESSING, and DONE). We use small conceptual implementations of the same design intent rather than line-for-line ports of the full D2S production modules.

4.2 Implementation paths

- Java: `Part3/code/java/Strategy.java, Observer.java`
- Python: `Part3/code/python/strategy.py, observer.py`
- JavaScript: `Part3/code/js/strategy.js, observer.js`
- C++: `Part3/code/cpp/strategy.cpp, observer.cpp`

We used the root README for sample compile/run commands. Our supplied verification notes record PASS for the examples. We treat these as archived manual verification claims because we did not rerun them here.

4.3 Metrics recorded in the CSV

Language	Pattern	LOC	Max CC	Median ms	Interfaces	Language note
Java	Strategy	45	2	12	1	Explicit interface, compile-time enforcement
Java	Observer	38	2	11	1	Observer list and explicit interface
Python	Strategy	30	2	8	1	ABC / duck-typing options
Python	Observer	28	2	7	1	Protocol/structural typing described in notes
JavaScript	Strategy	32	2	6	0	Dynamic contract / duck typing
JavaScript	Observer	30	2	5	0	forEach dispatch
C++	Strategy	48	2	3	1	Pure virtual abstraction / shared ownership
C++	Observer	42	2	3	1	Virtual destructor / RAII notes

Metric caveat: We recorded numeric timings and LOC in our CSV, but we could not find raw timing output, a measurement script, a clear LOC-counting artifact, or tool/version logs to verify the numbers. The instructions require us to compile the examples, perform one warm-up, and complete five measured runs on the same machine. Until we add that evidence, we label these values reported, not independently verified; we do not present them as freshly measured. We also need to normalize the supplied CSV schema to the instructor's required column names before final upload.

4.4 Comparison

In Java and C++, we can express explicit interface contracts and inheritance, which add boilerplate while providing stronger static constraints. In Python, we can use ABCs or protocols while keeping implementations concise. In JavaScript, we usually rely on object shape and runtime behavior rather than formal interface declarations. Across all four languages, we can use composition to store strategies or observers. We use the examples to demonstrate equivalent pattern intent; we do not treat LOC alone as a measure of design quality.

5 LLM Records and Human Verification

We found Part 1 prompt/input/output/verification files, a CH02 prompt/output note, and several Part 3 prompts/outputs in the archive. The root data/llm.csv summarizes five records. We preserve the supplied model/date claims (Claude Sonnet 4.6, 10 October 2026) as recorded metadata, not independently authenticated model logs. We found that some paths in llm.csv point to outputs rather than source inputs. We need to separate prompt, input, output, and verification paths according to the instructor schema and retain the exact raw outputs.

6 Findings, Contributions, Limitations and References

6.1 Findings

1. Repository/CRUD abstractions separate persistence operations from API handlers.
2. FastAPI dependency injection provides request-level services such as database sessions and user context.
3. Protected static-file serving wraps file streaming with access checks.

4. Policy rules and workflow listeners provide credible examples of Strategy and Observer-style collaborations in the supplied snapshot.
5. Cross-language implementations show how static contracts, dynamic dispatch and object ownership differ across Java, Python, JavaScript and C++.

6.2 Contribution record

We completed this case study as Team B15 and divided the work equally across the four members. We each contributed one quarter of the overall effort (25% each), including review, cross-checking, and final report preparation. We allocated the primary responsibilities as follows:

- Harikrishna T P (AM.SC.U4CSE23321): Source-repository review; environment and source-scope summary; architecture and request-flow documentation.
- Chandana CM (AM.SC.U4CSE23313): Pattern inventory; source-location evidence; Strategy analysis and policy-engine review.
- Pooja T (AM.SC.U4CSE23342): CH01–CH03 evolution review; evidence-gap checks; metrics and reproducibility notes.
- Rithu Mithra SS (AM.SC.U4CSE23345): Cross-language Strategy/Observer comparison; LLM-record review; diagram and report consistency checks.

We reviewed the combined report together and shared responsibility for the final submission. We record each member's contribution equally as 25% of the total work.

6.3 Limitations requiring action before submission

1. We need to add CH01 version model/CRUD/endpoints, CH03 changed task/UI files, exact diffs, and raw test logs, or revise the report to match the work actually present.
2. We need to establish a clean upstream baseline and reproducible commit/diff; our current Part 1 tree contains the policy engine and workflow pipeline.
3. We need to confirm whether the CSV's reported timings/LOC/complexity are measured, attach raw output, and document the counting rule.
4. We need to correct the required CSV headers and include source locations, participants, and status for each pattern instance.
5. We need to remove local .env files and any credentials from the submission archive/repository.

6.4 References

- Data-to-Science source repository: https://github.com/gdslab/data-to-science
- 23CSE455 Design Patterns Case Study Instructions and Case Study Report template, supplied PDFs.
- Gamma, Helm, Johnson and Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.
- FastAPI documentation: https://fastapi.tiangolo.com/
- React documentation: https://react.dev/
- Celery documentation: https://docs.celeryq.dev/

Appendix A — Supplementary Diagrams

This appendix adds four diagrams that we prepared for the case study: a component-level architecture view, a design-pattern class view, an upload-and-processing sequence, and a use-case view. They complement Figure 1 (architecture) and Figure 2 and Figure 3 (policy and workflow patterns) in the main report. Evidence status: these are our own reconstructions of D2S behaviour. We did not re-check the class and function names shown in them against a clean upstream checkout in this revision, so we treat them as supporting illustrations and not as source-verified evidence. Where a diagram label uses a pattern name with a qualifier such as "-like" or "-style", we mean an idiom that resembles the pattern, not a confirmed Gang of Four instance.

A.1 Diagram index and cross-reference

Figure	Diagram	What it shows	Related report section
5	Component architecture	Layered view from the React frontend through Nginx to FastAPI, CRUD, models and PostgreSQL/PostGIS, plus tusd, Varnish/TiTiler, Celery workers and file storage.	1.2 Architecture, 1.3 Request flow
6	Design-pattern class view	Six pattern-like collaborations: Repository, Dependency Injection, Proxy/Decorator-like static-file wrapper, Strategy-like backend selection, Interceptor-style token recovery, Observer-style auth events.	2.2 Pattern inventory
7	Sequence: upload to map view	Researcher upload, storage, asynchronous processing by a worker and external engine, result persistence, and later map retrieval with an access check.	1.3 Request and event flow
8	Use-case view	Actors (researcher/owner, collaborator, system administrator, external worker) and their use cases on the D2S platform.	1.1 Application, 2.3 Investigation areas

A.2 How the diagrams relate to the investigation areas

The sequence and use-case diagrams illustrate the researcher-management, dataset-management and experiment-management areas listed in Section 2.3: researchers upload raw UAV data, logs of flights and campaigns organise the work, and processing is dispatched to an external engine. The use-case view also shows project access and roles, which relates to the access-control area. The class view lists six idioms. Two of them (Repository and Dependency Injection) correspond to the first two inventory rows in Section 2.2; the others (protected static-file wrapper, backend selection, interceptor-based token recovery, and authentication events) are discussed with the qualifiers given there. Neither diagram set adds evidence for dataset versioning or for a complete notification centre, so the gaps recorded in Section 2.3 and Section 3 are unchanged.

A.3 Component architecture

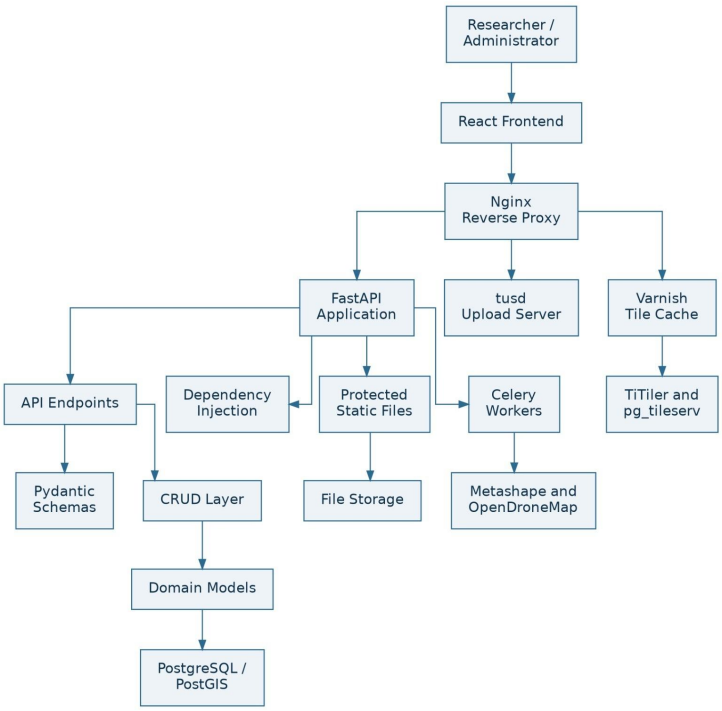


Figure 5: Layered component architecture of D2S as reconstructed by our team.

Requests pass from the React frontend to the Nginx reverse proxy, which routes to the FastAPI application, the tusd upload server and the Varnish tile cache. Inside FastAPI, endpoints use Pydantic schemas and the CRUD layer over domain models and PostgreSQL/PostGIS; dependency injection, protected static files and Celery workers (Metashape and OpenDroneMap) hang off the application. Varnish fronts TiTiler and pg_tileserv for tile delivery.

A.4 Design-pattern class view

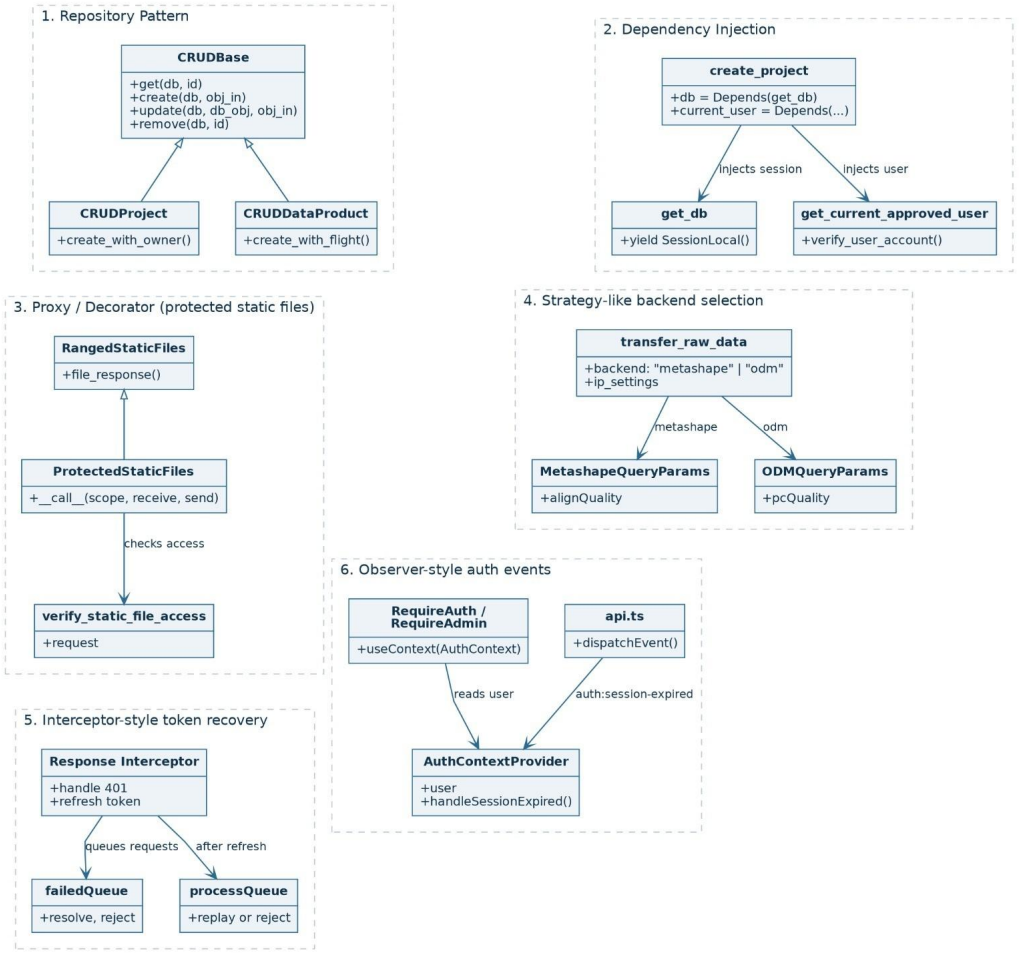


Figure 6: Six pattern-like collaborations in the D2S backend and frontend.

(1) CRUDBase with CRUDProject and CRUDDataProduct specialisations (Repository). (2) Endpoint functions receive the database session and the approved user through FastAPI Depends. (3) ProtectedStaticFiles extends RangedStaticFiles and calls verify_static_file_access before serving (Proxy/Decorator-like). (4) transfer_raw_data chooses between Metashape and ODM query parameters (Strategy-like selection, implemented as a conditional, not as an interface hierarchy). (5) A response interceptor queues failed requests and replays them after a token refresh. (6) api.ts dispatches an auth:session-expired event that AuthContextProvider handles (Observer-style).

A.5 Sequence: upload to processed-map view

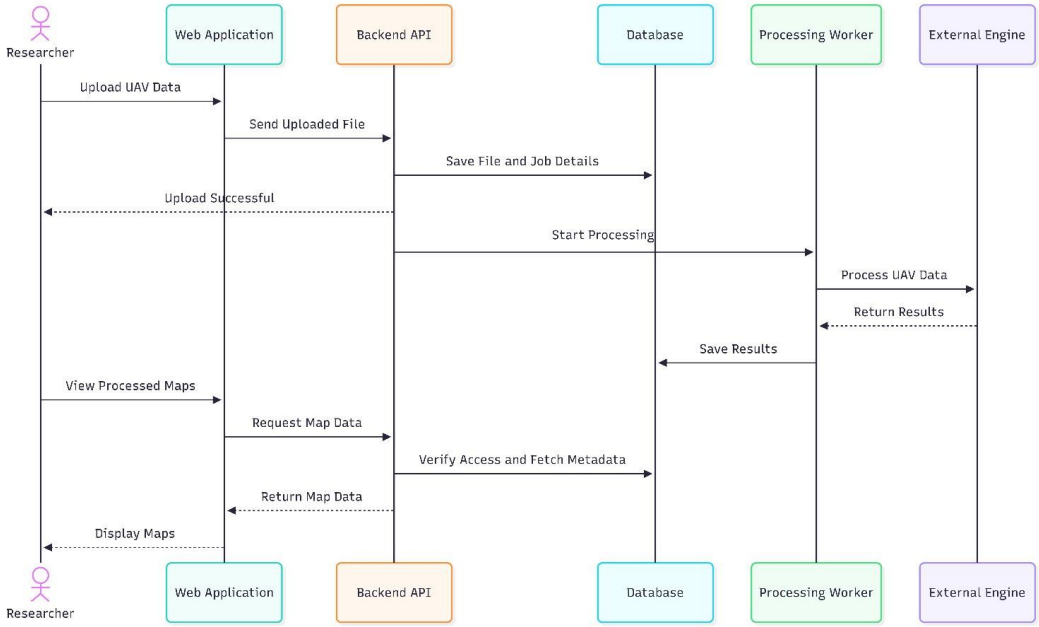


Figure 7: Upload, asynchronous processing and map retrieval.

The researcher uploads UAV data through the web application; the backend API stores the file and job details in the database and confirms the upload. Processing starts separately in a processing worker, which calls the external engine and saves the results. Later, when the researcher views maps, the backend verifies access and fetches metadata from the database before returning map data. The diagram shows that upload confirmation and processing are decoupled.

A.6 Use-case view

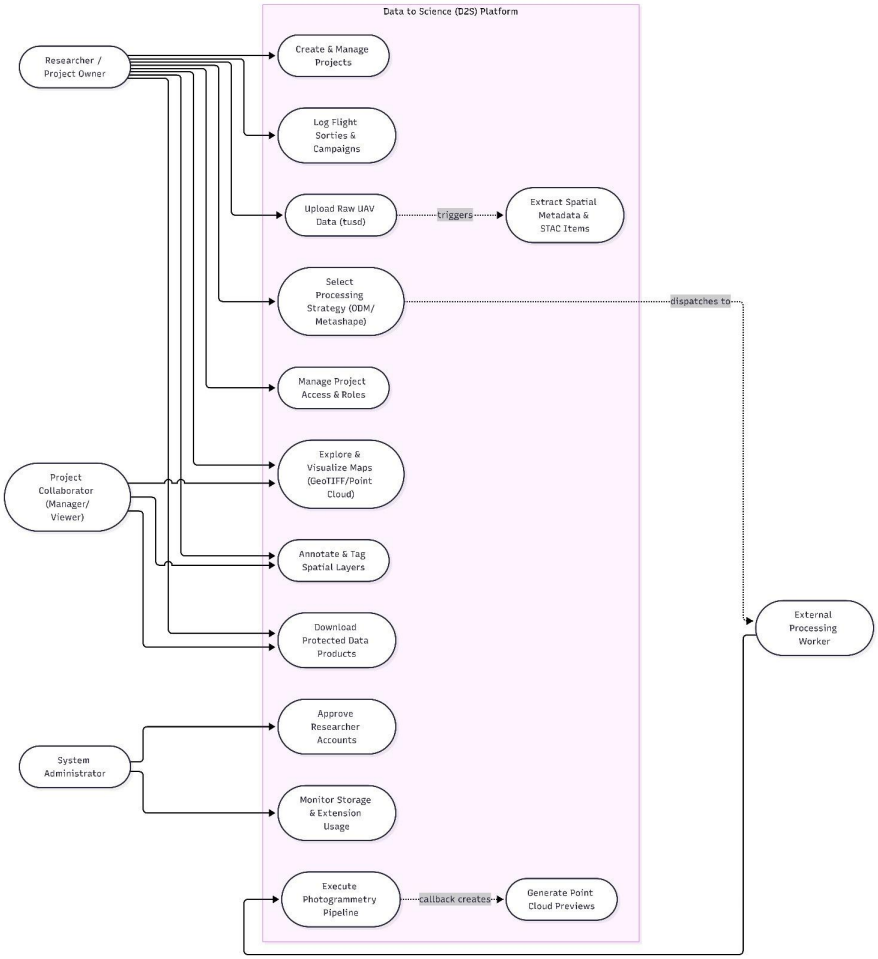


Figure 8: Actors and use cases of the Data to Science platform.

The researcher or project owner creates projects, logs flight campaigns, uploads raw data (which triggers spatial-metadata and STAC extraction), selects a processing strategy that dispatches to an external worker, and manages project access. Collaborators explore maps, annotate layers and download protected products. The system administrator approves researcher accounts and monitors storage and extension usage. The photogrammetry pipeline callback creates point-cloud previews.